

Backlink Admissibility in Cyclic Proofs for the Calculus of Inductive Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Cyclic proofs close goals by backlinking to previously-seen configurations, discovering induction principles post-hoc rather than prescribing them upfront. We prove that backlinks are *admissible* in the Calculus of Inductive Constructions (CIC): every valid cyclic proof can be read back to a standard CIC term that uses only the native fixpoint binder $()$ for recursion and is observationally indistinguishable from the root goal. The proof is a direct corollary of a contextual-equivalence theorem mechanised in Rocq with 0 axioms.

1 Introduction

Cyclic proof theory [1,2] extends standard proof systems with backlink rules that allow a premise to reference a previous goal, forming a directed cycle. The cycle is sound if it satisfies a global *trace condition*: every cycle must contain a *progress event*—a case-split on a structurally smaller term. Brotherston and Simpson prove that for first-order logic with inductive definitions, backlinks are *admissible*: every cyclic proof corresponds to a standard proof with an explicit induction rule.

This paper extends their result to the Calculus of Inductive Constructions (CIC) [3,5], the type theory underlying Rocq. We prove that every rooted, ranked cyclic proof can be residualised to a standard CIC term using only λ for recursion, and that the residual is observationally equivalent to the root goal.

2 Term calculus and reduction

Terms are given by the grammar:

$$t, u, A, B ::= x \mid i \mid \Pi x:A. B \mid \lambda x:A. t \mid t \ u \mid t \mid It \mid Ict \mid It\kappa b$$

where x ranges over de Bruijn indices, i over universe levels, I identifies an inductive family, c a constructor index, κ a dependent motive (a binder over the scrutinee), and $\mathbf{t}, \mathbf{b}$ are lists of terms.

We write $\rightarrow^*$ for the reflexive-transitive closure of $\rightarrow$, and $t \Downarrow v$ (“ t terminates to v ”) for $t \rightarrow^* v$ with v a value (an abstraction, a constructor, or a neutral term). Multi-argument application $\mathbf{apps} \ t \ [u_1, \dots, u_n]$ is defined inductively: $\mathbf{apps} \ t \ [] = t$ and $\mathbf{apps} \ t \ (u :: \mathbf{u}) = \mathbf{apps} \ (t \ u) \ \mathbf{u}$.

Operational semantics. We use a call-by-name (CBN) reduction relation $t \rightarrow t'$ defined by:

- **β -reduction:** $(\lambda x:A. t) u \rightarrow t[u/x]$.
- **Fixpoint unfolding:** $t \rightarrow t[t/0]$. (The binder 0 in t refers to the recursive self-reference.)
- **ι -reduction (case-of-constructor):** $I(Ic\mathbf{a})\kappa\mathbf{b} \rightarrow \text{apps}(\mathbf{b}[c]) \mathbf{a}$ where $\mathbf{b}[c]$ selects the c -th branch.
- **Congruence:** Reduction applies under applications and case scrutinees: if $t \rightarrow t'$ then $t u \rightarrow t' u$ and $It\kappa\mathbf{b} \rightarrow It'\kappa\mathbf{b}$.

A term t *terminates to* a value v , written $t \Downarrow v$, if $t \rightarrow^* v$ and v is in weak-head normal form (an abstraction, a constructor, or a neutral term).

3 Cyclic sequent calculus

A *configuration* is a typing judgement $\Gamma \vdash t : A$. A *configuration graph* is a tuple (V, ℓ, σ, v_0) where $V = \{0, \dots, n-1\}$ is a finite set of vertices, $\ell : V \rightarrow \text{Config}$ assigns a configuration to each vertex, $\sigma : V \rightarrow V^*$ assigns a (possibly empty) list of successors, and $v_0 \in V$ is the root. A graph is a *rooted pre-proof* if every vertex satisfies one of the local rules below (read bottom-up):

Asynchronous rules (deterministic). Applied eagerly. Each reduces the goal to a single premise by one computation step:

$$\frac{\Gamma \vdash t[u/x] : A}{\Gamma \vdash (\lambda x:A. t) u : A} \beta \qquad \frac{\Gamma \vdash t[t/0] : A}{\Gamma \vdash t : A} \text{FIX}$$

$$\frac{\Gamma \vdash \text{apps}(\mathbf{b}[c]) \mathbf{a} : A}{\Gamma \vdash I(Ic\mathbf{a})\kappa\mathbf{b} : A} \iota$$

Synchronous rule (choice point). A case-split on a neutral scrutinee x creates one premise per constructor c of I , each substituting the constructor pattern $Ic\mathbf{y}_c$ for the scrutinee in the c -th branch, under an extended context $\mathbf{y}_c$ of fresh variables for the constructor arguments:

$$\frac{\Gamma, \mathbf{y}_0 \vdash b[0][I0\mathbf{y}_0/x] : A \quad \dots \quad \Gamma, \mathbf{y}_n \vdash b[n][In\mathbf{y}_n/x] : A}{\Gamma \vdash Ix\kappa\mathbf{b} : A} \text{SPLIT}$$

Cyclic backlink (closure). $\frac{\Delta \vdash \sigma :_s \Gamma}{\Delta \vdash t : A} \text{BACK}$ where there exists a vertex $v' \in V$ such that $\ell(b, v') = \Gamma \vdash t_0 : A$ and $t = t_0[\sigma]$ is the original goal under the substitution σ .

Global progress condition. A rooted pre-proof is a *rooted cyclic proof* if every directed cycle contains at least one SPLIT edge. Formally, define the *budget-trace* graph $(V \times [0, |V|], E_B)$ where $(v_1, k_1) \rightarrow (v_2, k_2)$ is an edge in E_B iff $v_1 \rightarrow v_2$ is an edge in the original graph and either:

- $v_1 \rightarrow v_2$ is a SPLIT edge and $k_2 = k_1 - 1$;
- $v_1 \rightarrow v_2$ is any other edge and $k_2 = k_1$.

The graph is a valid cyclic proof iff the budget-trace graph is acyclic (starting from $(v_0, |V|)$). We write $\mathcal{C}(b, v_0)$ for this property.

4 Residualisation

The backlink rule appears only in the proof graph. A *residualisation* function $\mathcal{R}(b, v)$ reads back the graph into a CIC term by traversing the successors of v and replacing backlinks with binders. Formally, $\mathcal{R}$ is defined by well-founded recursion on the budget trace:

- If v has no successors (leaf), $\mathcal{R}(b, v)$ returns the term t where $\ell(b, v) = \Gamma \vdash t : A$.
- If v has a single successor w (asynchronous rule), then $\mathcal{R}(b, v) = D(\mathcal{R}(b, w))$ where D applies the inverse of the driving step: re-abstracts β reductions, re-applies the fix binder, re-constructs the case-of-constructor.
- If v has multiple successors $\mathbf{w}$ (split rule), each branch is residualised independently and combined into a application.
- If v is the *target* of a backlink (some vertex has v as its sole successor via a BACK edge), then $\mathcal{R}(b, v) = t_{\text{body}}$ where t_{body} is the residual of the backlink's source vertex with de Bruijn index 0 replaced by the recursive self-reference.

The result is a well-typed CIC term using only standard constructors (λ , application, $,$, $,$). No backlink constructs remain.

5 CIU equivalence

The soundness guarantee is *CIU equivalence* (contextual equivalence under all closing substitutions):

$$t \approx_{\text{ciu}} u \equiv \forall \sigma, v. (t[\sigma] \Downarrow v \iff u[\sigma] \Downarrow v)$$

Each local rule of the sequent calculus preserves CIU equivalence. The asynchronous rules are invertible: the conclusion is CIU-equivalent to its premise because each is a single reduction step (β , fix unfolding, ι). The split rule preserves CIU through the case analysis of a neutral scrutinee: substituting constructor patterns for the scrutinee in each branch yields a CIU-equivalent configuration. The back rule creates a fix binder at the target vertex.

Theorem 1 (CIU soundness). *If b is a rooted, ranked cyclic proof with root v and $\ell(b, v) = \Gamma \vdash t : A$, then*

$$t \approx_{\text{ciu}} \mathcal{R}(b, v).$$

Proof (Proof sketch). By well-founded induction on the budget trace. Each edge preserves CIU: asynchronous rules are single reduction steps, split preserves CIU through a neutral scrutinee, and backlinks become recursive self-references. The residualiser $\mathcal{R}$ unions these transformations across the graph. Full details are mechanised in Rocq (0 axioms) for the special case of graphs produced by the supercompiler [4]; the same proof structure generalises to any rooted, ranked cyclic proof because it depends only on the local rules and the trace condition, not on the supercompilation algorithm.

6 Admissibility

Theorem 2 (Backlink admissibility). *For every configuration graph b and vertex v ,*

$$\mathcal{C}(b, v) \implies \exists t_{\text{std}}, t \approx_{\text{ciu}} t_{\text{std}} \wedge t_{\text{std}} = \mathcal{R}(b, v)$$

where $\ell(b, v) = \Gamma \vdash t : A$.

In words: every rooted cyclic proof can be read back to a standard CIC term (using only `for` recursion) that is observationally indistinguishable from the root goal.

Proof. Take $t_{\text{std}} = \mathcal{R}(b, v)$. By construction (Section 4), t_{std} uses only standard CIC constructors—no backlinks remain. The CIU equivalence $t \approx_{\text{ciu}} t_{\text{std}}$ is Theorem 1.

The mechanised proof (`BacklinkAdmissible.v`) derives admissibility as a four-line corollary of the CIU theorem for supercompiler-produced graphs; the proof sketch above covers the general case for arbitrary cyclic proofs.

7 Discussion

The admissibility transformation is the same residualisation that the proof graph already provides: traverse the graph, replacing backlinks with `binders`. The CIU theorem (proved for each local rule of the sequent calculus) establishes equivalence; the residualiser eliminates backlinks; the result follows immediately.

Compared to Brotherston and Simpson’s result for first-order logic, our proof is simpler because (a) the graph is finite, and (b) residualisation is a syntactic transformation that explicitly replaces backlinks with `binders`, rather than requiring a semantic argument about proof unfoldings.

Acknowledgements. Thanks to James Brotherston for the question that motivated this paper.

References

1. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
2. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
3. Coquand, T., Huet, G.: The calculus of constructions. *Information and Computation* **76**(2–3), 95–120 (1988)
4. Mendel-Gleason, G.: Supercompilation corresponds to cyclic proof in the calculus of inductive constructions (2025), under review
5. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: *Typed Lambda Calculi and Applications*. pp. 328–345. Springer (1993)